SentinelFlow: Real-Time DDoS Detection and Adaptive Mitigation Using a Random Forest

MALLELA PALLAVI
Roll No. 23K91A05E3
Dept. of Computer Science and Engineering
[COLLEGE NAME]
[CITY], India
[EMAIL]

KATTOJU DURGABHAVANI
Roll No. 24K95A0514
Dept. of Computer Science and Engineering
[COLLEGE NAME]
[CITY], India
[EMAIL]

NIMMALA RAKSHITHA
Roll No. 23K91A05H3
Dept. of Computer Science and Engineering
[COLLEGE NAME]
[CITY], India
[EMAIL]

PRODHATOOR SHASHI PREETHAM
Roll No. 23K91A05K0
Dept. of Computer Science and Engineering
[COLLEGE NAME]
[CITY], India
[EMAIL]

V MEENAKSHI
[DESIGNATION]
(project guide)
Dept. of Computer Science and Engineering
[COLLEGE NAME]
[CITY], India
[EMAIL]

***Abstract*---**A denial-of-service flood is only half handled when it is detected: the service stays down until traffic is actually contained, and blocking legitimate users is itself an outage. This paper presents SentinelFlow, a working host-level system that captures traffic with Scapy or a built-in simulator, reduces each rolling window to ten flow features, classifies it with a 200-tree Random Forest, scores how widely the traffic is spread over sources and destinations, assigns an explainable severity, and quarantines the busiest sources for a limited time before they recover automatically. The shipped model was trained on a synthetic fallback and scores 99.7 % on its own split, but only 71.8 % on 225,745 real CICIDS2017 DDoS-day flows. Retrained on those flows, the same forest reaches 99.94 % over five seeds. The severity layer removes most false alarms, window length matters, and mitigation is application-level only. 95 automated tests pass; live capture was not tested.

***Index Terms*---**adaptive mitigation, CICIDS2017, DDoS detection, network security, Random Forest, Scapy.

I. INTRODUCTION

Distributed denial-of-service (DDoS) attacks exhaust a victim's bandwidth or processing capacity with traffic from many sources [2]. Rule and signature defences miss patterns they have not seen, and a variant called carpet bombing spreads the load over many destinations so that each flow looks small [3]. Machine-learning detectors are the usual answer, and recent surveys of detection and mitigation in software-defined networks (SDN) are extensive [4]. The cost of the usual practice is that evaluation stops at an offline accuracy figure: nothing is blocked, no recovery is shown, and the false-positive cost, a legitimate source quarantined, is never priced.

Four problems motivate the work. Signature-based defences cannot detect novel or modified attack patterns. Detection without mitigation leaves the service unavailable while the attack continues. Blocking legitimate users through false positives is itself a denial of service. And attack traffic must be classified quickly enough to act before resources are exhausted. A usable system therefore needs a classifier, a decision rule that is cautious about acting, an action that expires by itself, and a way for an operator to see why a decision was taken.

The base paper [1] builds an ensemble for SDN detection from Random Forest feature importance combined with a Banzhaf power index, compares seven classifiers and four voting ensembles on two datasets, reports 100 % on both and profiles training time, latency and model size; live deployment and mitigation are left as future work. The gap this project addresses is the closed loop from real-time detection to adaptive mitigation with a measured response.

The contributions are: (1) SentinelFlow, a tested end-to-end product with a React console, twelve JSON endpoints and a SQLite history; (2) an explainable distributed-risk and severity layer that lists its contributing factors and cuts false alarms; (3) a temporary, self-recovering quarantine with a safe Simulation Mode that drives the identical pipeline; and (4) an evaluation that separates synthetic from real data and states the gap. Section II surveys related work, Section III gives the method, Section IV the results and Section V concludes.

A. System Hardware

All measurements ran on one CPU-only desktop: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11, no GPU. Other jobs kept the processor at 100 % during the latency runs, so the timings below are upper bounds. Npcap was not installed.

B. Software Requirements

Python 3.11, Flask 3.1.3 [5], scikit-learn 1.9.1 [6], pandas 3.1.0, NumPy 2.4.6, psutil 7.2.2, Scapy 2.8.0 [7] and SQLite; React 19 with Vite 8 for the console; pytest for the tests. Live capture needs Npcap and Administrator rights on Windows.

II. LITERATURE SURVEY

Base paper. Trigui et al. [1] select features with Random Forest importance and a Banzhaf-index perturbation score, benchmark XGBoost, Random Forest, Gradient Boosting, SVM, kNN, AdaBoost and Naive Bayes, and combine the best learners in four hard-voting ensembles. They use an SDN flow dataset of 104,345 records with 23 features and a 1,754,109-row, 47-feature subset of CIC-DDoS2023. All four ensembles reach 100 % accuracy, precision, recall and F1 on both. The resource profile is the paper's strongest point: the two SVM ensembles needed more than 4,000 s to train on the SDN data and the kNN ensemble more than 150 s to classify its test set, while the lightest ensemble was small and fast.

ML detection in SDN. Alashhab et al. [8] use an online ensemble for detection and mitigation; Zaman et al. [9] study

decision trees; Alduailij et al. [10] combine mutual information with forest importance and report 99 % on CICIDS2017 and CIC-DDoS2019, and Becerra-Suarez et al. [11] report 99.97 % accuracy (both as quoted in the base paper). Raghavendran and Robinson [12] report 97 % for kNN and 100 % for XGBoost on a testbed dataset, and Rahman [13] reports 97.1 %, 82.7 % and 59.3 % for SVM, decision tree and logistic regression on a CIC sample. Three further survey items [14], [15], [16] (one uses double deep Q-networks) are cited by title only, as their abstracts were unavailable.

Carpet bombing and dataset caution. Swileh et al. [3] detect carpet-bombing traffic in SDN with retrieval-augmented generation and a large language model (99.94 % accuracy, about 2 s per request, no supervised training). Two warnings apply to every number in this field: CICIDS2017 [17] has labelling and flow-construction flaws [18], and detectors tuned on one closed dataset often fail in operation [19].

TABLE I
SELECTED RELATED WORK

Work	Approach	Data	Reported result	Limitation
Base paper [1]	RF-BPI selection, 4 voting ensembles	SDN 104,345; CIC-DDoS2023 1.75 M	100 % on both	SVM/kNN costly; no mitigation
Swileh [3]	RAG + FAISS + LLM	Mininet/Ryu, hping3	99.94 %, ~2 s/request	Needs an LLM service
Raghavendran [12]	KNN, XGBoost, NB	WISNE-SDN testbed	97 % (KNN), 100 % (XGB)	Testbed; detection only
Rahman [13]	LR, tree, SVM	CIC sample	SVM 97.1 %	Detection only
Alduailij [10]	MI + RF importance	CICIDS2017, CIC-DDoS2019	99 % (via base)	Older datasets
Becerra-Suarez [11]	Trees + deep nets	Public DDoS data	99.97 % (via base)	No resource profile
Engelen [18]	Dataset audit	CICIDS2017	Label flaws found	No detector
This work	RF + risk + severity + quarantine	Synthetic; CICIDS2017 DDoS day	Section IV	No live capture test

Mitigation in the literature. Mitigation is usually described as throttling or blackholing flows through SDN rules, or as ingress filtering against spoofed sources, and is rarely evaluated beyond a description. Where a response is evaluated it is tied to a controller or a testbed switch, which a single host does not have. The practical lesson for a host-level tool is conservative: act on the strongest evidence, expire the action, and never touch addresses that carry the operator's own traffic.

Positioning. The works above run in SDN controllers or testbeds and judge themselves by offline accuracy; the base paper adds resource profiling, the carpet-bombing study adds a realistic attack pattern. SentinelFlow works at host level on a single machine, uses a deliberately small feature set that both a packet sniffer and a flow dataset can supply, and puts its effort into what happens after the label: how confident to be, what to do, for how long, and what to show the operator. That choice trades the scale of an SDN deployment for a loop that can be run, inspected and tested on a laptop.

Research gap. Reported accuracies of 99-100 % come from offline runs on one dataset. Missing are a capture-to-action loop on a commodity host, a score that flags carpet-bombing spread, and an evaluation that prices false positives at the level where they hurt and says how far a synthetic training set carries.

III. METHODOLOGY

A. Data and Features

Each monitoring window is reduced to ten features: received and sent packets, bytes, packets per second, bytes per second and average packet size. Packets whose destination is a local address are received, those from a local address to a remote one are sent, and transit traffic counts as received. The same schema is built from CICIDS2017 rows [17]: forward packets and bytes become sent, backward become received, rates are counts over flow duration. The training script looks for CICIDS2017 CSV files in data/raw; none was present when the product was built, so it generated a labelled synthetic fallback of 9,000 benign and 9,000 attack flows and added 6,000 windows from the simulator aggregated through the real monitoring pipeline: 19,200 training and 4,800 test rows. For this paper we also downloaded the CICIDS2017 Friday-afternoon DDoS file from a public mirror: 225,745 flows, 97,718 benign and 128,027 DDoS.

Cleaning follows the product's preprocessing module: column names are normalised, infinite values are dropped, every label other than BENIGN becomes Attack, missing packet or byte columns are treated as zero, flow durations are clipped to 1 microsecond-1 hour before rates are computed, and averages are zero when a direction has no packets. The simulator that produces demonstration and augmentation windows has four traffic styles: benign mixes of 60-600 packets between one local host and a few remote servers; floods of 2,500-9,000 small packets from 8-40 sources to one host; carpet-bombing windows of 1,800-6,000 packets from 10-45 sources to 6-20 victims; and slow windows of 300-1,200 TCP packets from many sources.

B. Classifier

A Random Forest [20] of 200 trees with balanced class weights and seed 42 was chosen because it handles skewed, non-linear count features without scaling, trains in seconds, gives importances for the report and produces a probability that serves as confidence. Given a feature vector x, the forest votes with probability

$$p(\text{attack} \mid x) = (1/T)\ \Sigma_t\ 1[\ \text{tree}_t(x) = \text{attack}\ ], \quad T = 200 \tag{1}$$

C. Distributed Risk and Severity

A transparent rule layer measures how distributed the window is. With n_s unique sources, n_d destinations, packet rate r and the mean $\bar{r}$ of the last twelve windows:

$$R = 0.30\ \min(n_s/30, 1) + 0.30\ \min(n_d/15, 1) + 0.25\ \min(r/1500, 1) + 0.15\ \min(\max(r/\bar{r} - 1, 0)/4, 1) \tag{2}$$

The severity score S adds fixed points for an Attack label (30), confidence of 0.9 or more (15), packet rate (up to 20), R of 0.7 or more (20), a wide spread (8) and persistence over consecutive suspicious windows (up to 10); benign windows are capped at 45. Low, Medium, High and Critical start at 0,

28, 50 and 70, and every contributing factor is stored as text. High and Critical windows quarantine the three busiest non-protected sources for T_q = 40 s (80 s for Critical); Medium only recommends rate limiting; expiry marks the record recovered.

Two safeguards follow from the cost of false positives. Loopback, the simulator's own host and every local interface address are protected and never quarantined; if no unprotected source remains the system only recommends rate limiting. A quarantined address cannot be quarantined again while its record is active, so a long attack produces one record per source, not one per window. The persistence counter rises by one for every consecutive window that the forest calls Attack or whose R is at least 0.7, and resets on the first clean window.

D. Evaluation Protocol

Accuracy, precision, recall, F1 and ROC-AUC are computed from the confusion matrix with the attack class as positive; the false-positive rate is

$$FPR = FP / (FP + TN) \qquad (3)$$

because an ill-judged quarantine is the cost that matters most here. Three questions are asked separately: how the shipped model performs on its own held-out split, how it performs on real flows it never saw, and how the whole pipeline behaves on windows of 2, 5 and 10 s. A chronological block split (first 70 % of the file's rows for training, last 30 % for testing) and five random seeds check that the real-data result does not depend on one lucky split; a feature-group ablation and five alternative classifiers check what the 10 features carry.

E. Architecture and Implementation

A Flask service holds one function, process_window(), which both the Scapy engine and the simulator call, so live and simulated windows take the same path [5]. Detections, quarantines, health samples and settings go to SQLite; a background thread samples CPU and memory every five seconds; the React console polls every two seconds. The operating-system firewall is never changed: quarantine is a denylist inside SentinelFlow.

The REST layer exposes status, health, interfaces, detections (with a limit that is clamped to 1-500), the latest detection, mitigations, settings (window 2-120 s, quarantine 5-3600 s), start and stop for live capture and for simulation, and the stored model metrics; bad input returns HTTP 400, a second start returns 409 and an unknown path returns a JSON 404. The console shows a banner whenever the data are simulated and a separate warning on the Model screen when the model was trained on synthetic data. Access to the console is a browser-side demo sign-in; the API itself has no authentication and binds to 127.0.0.1 only.

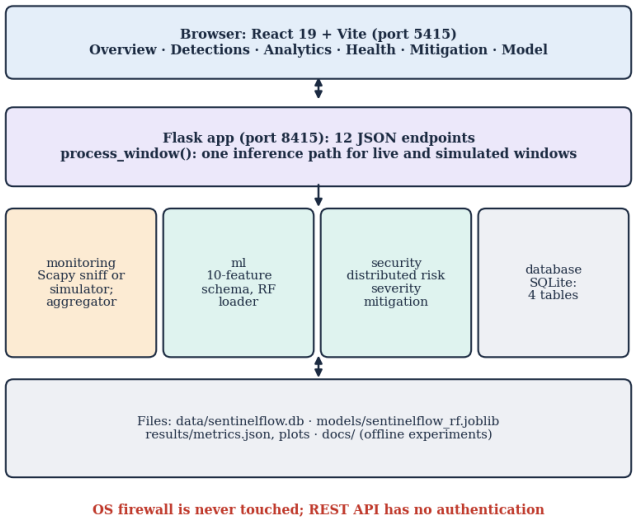


Fig. 1. SentinelFlow architecture: browser, Flask API, pipeline modules and local files.

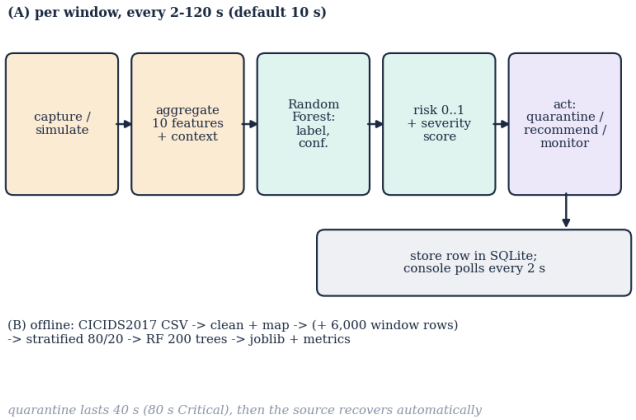


Fig. 2. Main flow: (A) one pass per monitoring window and (B) offline training.

IV. RESULTS

A. Test Conditions

Metrics were computed with scikit-learn on a stratified 80/20 split (seed 42; five seeds where stated). The window experiments used 4,000 windows per length from the product's own simulator generator (35 % attack, seed 2026), aggregated and scored statefully as in the application. 95 pytest tests (scratch database) and 25 backend smoke checks passed. Timings are on the saturated host of Section I-A.

TABLE II
DETECTION RESULTS (POSITIVE CLASS = ATTACK)

Setting	Acc. %	Rec. %	FPR %	F1 %
Shipped RF, own synthetic split (4,800)	99.71	99.46	0.04	99.71
Shipped RF, real DDoS-day flows (225,745)	71.8	63.4	17.2	71.8
RF retrained on real flows (5 seeds)	99.94±0.01	99.93±0.01	0.06±0.01	99.94±0.01
Retrained, first 70 % train / last 30 % test	99.74	99.77	0.28	99.67
Product recipe (real + window rows), mixed test	99.93	99.91	0.04	99.94

B. Real Data Against Synthetic Data

The shipped model's 99.71 % is a property of its synthetic data. On real flows it scores 71.8 % accuracy, 63.4 % recall and 17.2 % false-positive rate (ROC-AUC 86.1 %: the ranking is useful but the 0.5 threshold is not): 46,860 of 128,027 DDoS flows were missed. Retrained on real flows with the same ten features, 200 trees and settings, the forest reaches 99.94 % over five seeds and 99.74 % on a chronological block split, in 7 s with a 1.24 MB model. On the same split a decision tree scores 99.94 %, gradient boosting 99.95, logistic regression 98.6 and Gaussian Naive Bayes 72.9 %, so the forest is not special on this file. A feature ablation shows sent-side features alone give 99.89 % and received-side features alone 95.8 %. The model also shifts what it relies on: the shipped forest ranks sent and received packet counts first, the real-data forest ranks average sent packet size first.

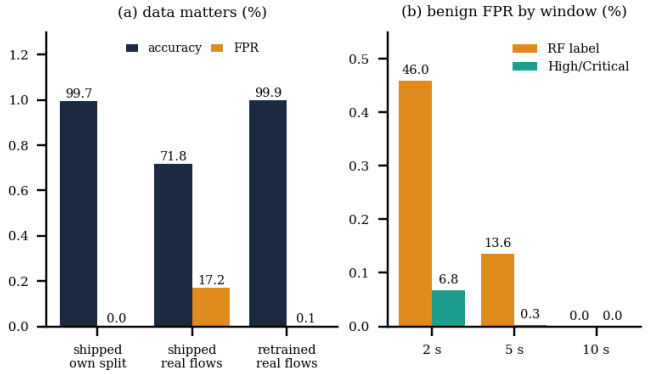


Fig. 3. (a) Accuracy and false-positive rate by training data; (b) false-positive rate on benign simulator windows by window length, RF label alone and High/Critical severity.

C. Window Length and the Severity Layer

The model was calibrated on 10 s windows. At 10 s the forest is perfect on simulator windows, but only because the training augmentation came from the same generator; this is not evidence about real attacks. At shorter windows packet rates rise and benign traffic looks like attack: the label alone raised 46.0 % false positives at 2 s and 13.6 % at 5 s. Acting only on High or Critical severity gives 6.77 % and 0.27 % with attack recall 100.0 % and 100.0 %. At 10 s that gate had 0.0 % false positives but missed 44 of 1,445 attack windows (recall 97.0 %), which stayed at Medium. The distributed-risk score on its own is precise but weak: at a threshold of 0.7 it flagged no benign window yet found only 9 % of attacks at 10 s and 32 % at 2 s; it helps inside the severity score, not as a detector.

TABLE III
PIPELINE ON 4,000 SIMULATOR WINDOWS PER LENGTH (35 % ATTACK)

Window	RF acc. %	RF FPR %	Sev. acc. %	Sev. FPR %	Sev. rec. %
2s	70.6	46.0	95.7	6.77	100.0
5s	91.3	13.6	99.8	0.27	100.0
10s	100.0	0.0	98.9	0.00	97.0

By traffic style at 5 s, floods averaged 1136 packets/s, carpet-bombing windows 780, slow windows 149 and benign windows 65; every attack window of every style reached High or Critical, and of the 2,555 benign windows 340 were Medium, 7 High and none Critical. Slow attacks are the weakest case: 499 of 500 reached only High, because their packet rate (about 149/s) earns few rate and spread points and they depend on the model label and persistence. In a 2 s live-simulation run through the real HTTP API, the first detection was stored 4.6 s after Start; 13 windows were scored (10 Attack, 3 Benign) and 21 quarantine rows were written.

D. Speed and Interface

From a finished feature vector to the mitigation decision took a median of 217 ms (95th percentile 489 ms) on the saturated host; aggregation was 1.0 ms and risk plus severity 0.03 ms. Most of the rest is forest inference: the saved model uses n_jobs=-1, which costs 313 ms per single window against 33 ms with one worker. The real delay is the window itself (default 10 s). Over HTTP on 127.0.0.1 the status endpoint answered in 3.3 ms median and the 500-row detections list in 5.9 ms. All 25 malformed-input probes (bad limits, out-of-range settings, path traversal, a second start) were refused or tolerated as intended and the service stayed healthy.

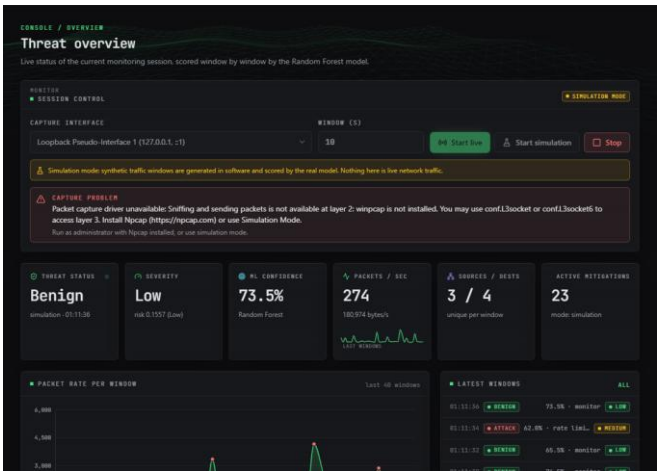


Fig. 4. Overview screen in Simulation Mode: session control, the real capture-driver warning (Npcap absent), status cards and the packet-rate history.

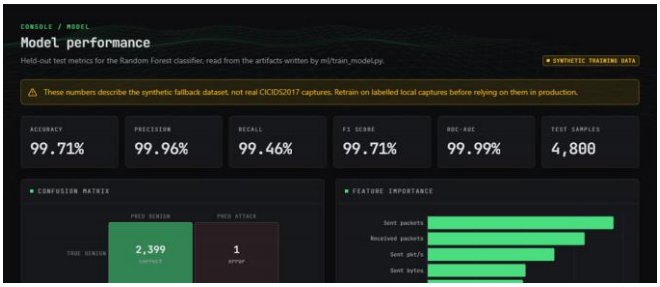


Fig. 5. Model screen: held-out metrics with the “synthetic training data” warning, confusion matrix and feature importance.



Fig. 6. Threat Detail screen for one scored window (Simulation Mode): verdict, measured traffic, distributed-risk components and severity factors.

Figs. 4-6 show the operator's view. The Overview screen labels the session as simulated and shows the real capture-driver warning; the Model screen carries the synthetic-data warning next to its 99.71 % accuracy; the Threat Detail screen lists, for one window, the measured traffic, the four components of R and every severity factor, together with the recommended action, so that a quarantine can be traced to its causes. These screens were captured from the running application with data scored by the real model at 2 s windows, and the Analytics screen of that session counted 155 of 237 windows as Attack although the simulator draws only 35 % attack windows, consistent with the 2 s false-positive rate above.

E. Threats to Validity

Four threats limit these numbers. First, the real-data result rests on one capture day and one attack type, so it says nothing about other attacks; random splits of flow data let neighbouring flows of one attack appear on both sides, which the block split only partly answers. Second, CICIDS2017 labels are known to contain errors [18]. Third, the window results come from a simulator whose generator also produced part of the training rows, so they test calibration, not generalisation. Fourth, no live capture, real service outage or real carpet-bombing traffic was available, so detection-to-mitigation time is a decision latency, not the time a victim would wait.

F. Comparison with the Base Paper and Failure Cases

The base paper's 100 % [1] is on a 104,345-row SDN dataset and a 1.75 M-row CIC-DDoS2023 subset with ensembles and Banzhaf-selected features; our 99.94 % is on CICIDS2017 DDoS-day flows with ten features and a single forest. The datasets, labels and protocols differ, so the figures are indicative only, and the CICIDS2017 file contains one attack type. On cost, our forest trains in seconds and the model is small, in line with the paper's lightest ensemble; we did not reproduce its Banzhaf selection or voting ensembles. Mitigation, which the paper leaves as future work, is implemented here only at application level.

Failures observed: the shipped model misses 37 % of real DDoS flows; the label alone is unusable at 2 s windows; live capture without Npcap returns the driver error through the status endpoint and stops the session; the interface setting accepts a 5,000-character string; the landing page shows the synthetic 99.71 % figure without a caveat; and quarantine drops no real packets.

V. CONCLUSION

SentinelFlow closes the detection-to-mitigation loop on a commodity host with an explainable severity layer, a self-recovering quarantine, a SQLite history and a React console, and it passes 95 tests. Of the six objectives, dataset handling, classifier, dashboard and evaluation were met; live capture is implemented but untested here, and mitigation is a denylist rather than enforcement. The key finding is a data gap: 99.7 % on synthetic data became 71.8 % on real flows, while the same forest retrained on real flows reached 99.94 %. Limits are one attack type, flow-level data, label noise, a simulator-driven demonstration and a CPU-saturated host for timings.

TABLE IV
OBJECTIVES AGAINST OUTCOMES

Objective	Outcome
Dataset and features	Partly: shipped model synthetic; real file mapped and tested
Live capture, same features	Implemented; not tested live (no Npcap)
Train classifier	Met on real flows after retraining (99.94 %)
Adaptive mitigation	Partly: self-recovering quarantine; no enforcement
Dashboard	Met: nine-page React console
Accuracy, FPR, time to mitigation	Met for accuracy, FPR; latency is decision time

Future work: retrain on the full CICIDS2017 and CIC-DDoS2023 data and test across datasets; enforce quarantine through the operating-system firewall or an SDN controller with explicit opt-in; test live capture and real carpet-bombing traffic on a testbed; set n_jobs=1 and window-aware calibration; add authentication to the API; and compare Banzhaf selection and voting ensembles on the same flows.

REFERENCES

[1] M. M. Trigui, M. S. Islam, M. D. Gambo, M. S. Alam and T. Helmy, "Resource-aware ensemble framework for DDoS detection in SDN via novel game-theoretic feature selection," *IEEE Access*, vol. 14, pp. 1666-1676, 2026, doi: 10.1109/ACCESS.2025.3618519.

[2] J. Mirkovic and P. Reiher, "A taxonomy of DDoS attack and DDoS defense mechanisms," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 34, no. 2, pp. 39-53, Apr. 2004, doi: 10.1145/997150.997156.

[3] M. N. Swileh, S. Zhang and K. Lei, "Intelligent detection and mitigation of carpet-bombing DDoS attacks in SDN using retrieval-augmented generation and large language models," arXiv:2605.26307, May 2026.

[4] Y. Su, D. Xiong, K. Qian and Y. Wang, "A comprehensive survey of distributed denial of service detection and mitigation technologies in software-defined network," *Electronics*, vol. 13, no. 4, p. 807, Feb. 2024, doi: 10.3390/electronics13040807.

[5] Pallets Projects, "Flask: A lightweight WSGI web application framework," documentation. [Online]. Available: https://flask.palletsprojects.com

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[7] P. Biondi *et al.*, "Scapy: Interactive packet manipulation program," software documentation. [Online]. Available: https://scapy.net

[8] A. A. Alashhab, M. S. Zahid, B. Isyaku, A. A. Elnour, W. Nagmeldin, A. Abdelmaboud, T. A. A. Abdullah and U. D. Maiwada, "Enhancing DDoS attack detection and mitigation in SDN using an ensemble

online machine learning model," *IEEE Access*, vol. 12, pp. 51630-51649, 2024, doi: 10.1109/ACCESS.2024.3384398.

[9] A. Zaman, S. A. Khan, N. Mohammad, A. A. Ateya, S. Ahmad and M. A. ElAffendi, "Distributed denial of service attack detection in software-defined networks using decision tree algorithms," *Future Internet*, vol. 17, no. 4, p. 136, Mar. 2025, doi: 10.3390/fi17040136.

[10] M. Alduailij, Q. W. Khan, M. Tahir, M. Sardaraz, M. Alduailij and F. Malik, "Machine-learning-based DDoS attack detection using mutual information and random forest feature importance method," *Symmetry*, vol. 14, no. 6, p. 1095, May 2022, doi: 10.3390/sym14061095.

[11] F. L. Becerra-Suarez, I. Fernandez-Roman and M. G. Forero, "Improvement of distributed denial of service attack detection through machine learning and data processing," *Mathematics*, vol. 12, no. 9, p. 1294, Apr. 2024, doi: 10.3390/math12091294.

[12] N. Raghavendran and Y. H. Robinson, "IoT-based WISNE-SDN detection and DDoS attack mitigation using machine learning techniques," *PeerJ Comput. Sci.*, vol. 12, Art. no. e3572, Feb. 2026, doi: 10.7717/peerj-cs.3572.

[13] M. A. Rahman, "Detection of distributed denial of service attacks based on machine learning algorithms," arXiv:2502.00975, Feb. 2025.

[14] S. Areef and G. Vijayasri, "DDoS attack detection and mitigation," *Int. J. Eng. Sci. Adv. Technol.*, vol. 26, no. 4, pp. 115-120, 2026, doi: 10.64771/ijesat.2026.v26.i6.3198 (record as given by Crossref; abstract not available).

[15] M. Jadric and S. Mladenovic, "DDoS attack detection and mitigation using double deep Q-networks," in *Proc. 2025 Int. Conf. Software, Telecommun. Comput. Netw. (SoftCOM)*, Split, Croatia, Sep. 2025, pp. 1-6, doi: 10.23919/SoftCOM66362.2025.11197364.

[16] "AI-driven DDoS attack detection and mitigation in SDN," *IJARCCE*, vol. 14, no. 4, 2025, doi: 10.17148/IJARCCE.2025.144107 (authors not listed in Crossref; abstract not available).

[17] I. Sharafaldin, A. Habibi Lashkari and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Funchal, Portugal, Jan. 2018, pp. 108-116, doi: 10.5220/0006639801080116.

[18] G. Engelen, V. Rimmer and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. 2021 IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 7-12, doi: 10.1109/SPW53761.2021.00009.

[19] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. 2010 IEEE Symp. Secur. Privacy*, 2010, pp. 305-316, doi: 10.1109/SP.2010.25.

[20] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, Oct. 2001, doi: 10.1023/A:1010933404324.